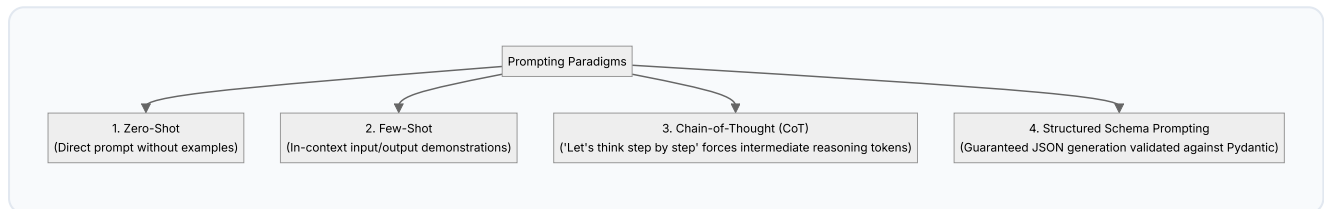


Agentic AI Chapter 2: Prompt Engineering, Tool Calling & Structured Outputs

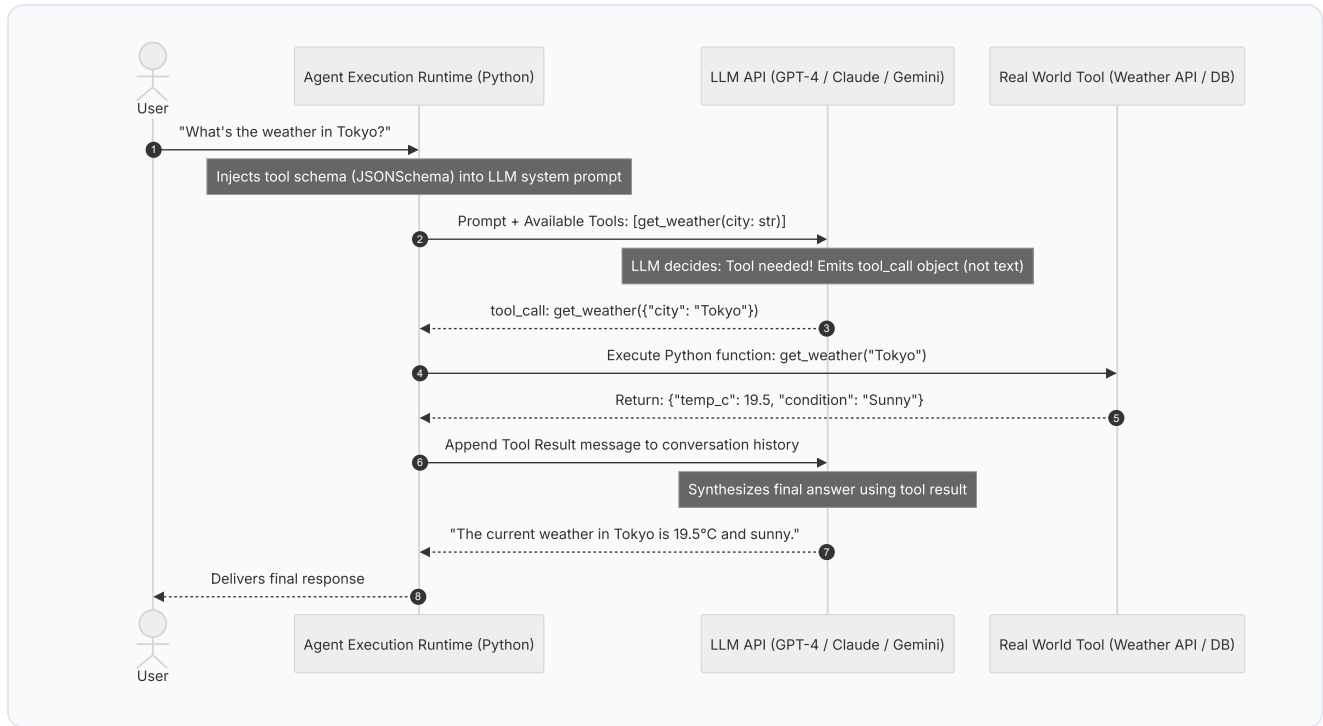
Core Learning Objective: Move beyond simple text generation. Learn how LLMs interface with the external world through the Function Calling protocol, JSONSchema validation, and Pydantic v2 structured outputs.

1. The Prompt Engineering Hierarchy



2. The Tool Calling (Function Calling) Protocol Under the Hood

Tool calling transforms an LLM from a passive text generator into an **autonomous computational agent**. The LLM itself does **not** execute the code; it acts as a **smart orchestrator that decides which function to call and formats valid JSON arguments**.



3. Tool-Calling Implementation in Python (Teaching Version)

Here is a complete, runnable in-memory tool loop implementing function calling without third-party dependencies:

4. Structured Output Enforcement with Pydantic v2

Modern enterprise systems forbid free-form text when extracting entities or generating database records. Pydantic v2 guarantees schema adherence:

Verified Worked Example: Tool Errors the Model Can Fix

In `examples/ex01_tool_loop.py` the `get_history` tool raises `ValueError("days must be between 1 and 7")`. The harness converts it into the observation `ERROR: days must be between 1 and 7`, which is what lets the model retry with a valid argument. **Design rule:** an error message is part of the tool's interface; say what was wrong and what valid input looks like.

Real-SDK caveat. In `mcp=2.3.0`, an exception raised inside an MCP tool reaches the model only as "Error executing tool " (the detail is not forwarded). For fixable mistakes, return the explanation in the result text instead (see [examples/ex04_mcp_server.py](#)). Verify how *your* SDK version reports tool errors before relying on them.

Pinned for the verified examples: `langgraph=1.2.13`, `mcp=2.3.0`, `pytest=9.1.1` (see [examples/requirements.txt](#)). All examples run offline with a scripted fake model: `cd examples && pip install -r requirements.txt && pytest -q`.

5. Runnable Model: Validating Tool Calls and Containing Untrusted Output

The model proposes a call; your code decides whether to run it. Three mechanical controls belong in every tool loop.



What this proves and what it does not: the validator guarantees that **only allowed calls with well-formed arguments execute**, whatever the model was tricked into proposing; the wrapper keeps hostile text from impersonating your own structure. It does **not** stop the model from being influenced by the text, which is why the allowlist (a permission) matters more than the wrapper (a hint).

Prompt and tool-description quality checklist

1. Describe each tool in terms of **when to use it and when not to**, with one example; vague descriptions cause wrong-tool calls more than weak models do.
2. Make argument names self-explanatory and **constrain** them (enums, ranges, formats) so invalid values are impossible to express.
3. Return **actionable errors** ("city not found, try a larger nearby city") rather than stack traces; the model can recover from a good message.
4. Keep the tool list **small and relevant** per request; dozens of tools degrade selection accuracy and burn context.
5. Version prompts and tool schemas together, and run the evaluation set when either changes.

Further Reading

- [Anthropic: tool use overview](#)
- [OpenAI: function calling](#)
- [Lilian Weng: prompt engineering](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. What is the difference between a system prompt and a user prompt?
- ▶ 2. In function calling, who executes the function?
- ▶ 3. Why give tool parameters enums and descriptions?
- ▶ 4. What is few-shot prompting?